

Universidad Tecnológica de Puebla

Tecnologías de la Información, área Desarrollo de Software Multiplataforma

Materia: Desarrollo de Aplicaciones Web Progresivas

SUBPRODUCTO 6

Notificaciones Push

Proyecto: AlertaVecinal, red de seguridad vecinal (PWA)

Profesora: [NOMBRE_PROFESORA]

Grupo: 10 D

Integrantes:
Serrano Arana Jahir Jaziel · Matrícula [MATRICULA_1]
Xicali Diaz Luis Alberto · Matrícula [MATRICULA_2]
Gonzalez Barrios Yeudiel Nain · Matrícula [MATRICULA_3]

Fecha: 29 de octubre de 2026

Índice

1. ¿Qué son las notificaciones push en una PWA? 3

2. Características 4

3. Ventajas y limitaciones 5

Ventajas 5

Limitaciones 5

4. Aplicaciones prácticas 6

5. ¿Cómo funciona por dentro? 7

6. Componentes y tecnologías necesarios 8

7. Ejemplo implementado en AlertaVecinal 9

7.1 Pedir permiso, obtener el token y guardarlo 9

7.2 Primer plano: onMessage 10

7.3 Segundo plano: onBackgroundMessage 10

7.4 Clic en la notificación: notificationclick 11

7.5 Envío desde la terminal: enviar-push.js 12

7.6 Pasos para enviar una notificación 13

7.7 Capturas 13

8. Conclusión 18

Referencias 19

URL de la aplicación 20

1. ¿Qué son las notificaciones push en una PWA?

Una **notificación push** es un mensaje que un servidor envía a un dispositivo y que el sistema operativo muestra como aviso (en la barra de notificaciones del celular o en una esquina de la pantalla de la computadora), **aunque la aplicación no esté abierta**. En una PWA, el que recibe el mensaje es el service worker, que vive en segundo plano.

En realidad son dos tecnologías que trabajan juntas:

- **Push:** el servidor *empuja* un mensaje hacia el navegador del usuario (Push API).
- **Notificación:** el service worker *muestra* ese mensaje en pantalla (Notifications API).

Su **propósito** es avisar a tiempo de algo importante sin que el usuario tenga que estar revisando la app. En AlertaVecinal es la función más importante: si un vecino reporta un robo o una fuga de gas, los demás deben enterarse en ese momento, no la próxima vez que abran la página.

2. Características

Característica	Explicación
Llegan con la app cerrada	El navegador despierta al service worker cuando llega el mensaje, aunque no haya ninguna pestaña abierta.
Requieren permiso	El usuario debe aceptar con `Notification.requestPermission()`. Se recomienda pedirlo con un botón y no al cargar la página.
Necesitan HTTPS	Solo funcionan en sitios seguros (o en *localhost* durante el desarrollo), igual que el service worker.
Necesitan un service worker	Es quien recibe el evento `push` y llama a `showNotification()`.
Un token por navegador	Cada navegador o dispositivo suscrito tiene su propio identificador (token de registro) para recibir mensajes.
Siempre visibles	En la web no se permiten pushes "silenciosos": cada mensaje debe mostrar una notificación al usuario.
Mensajes pequeños	El contenido del mensaje es de pocos kilobytes (alrededor de 4 KB en FCM): título, texto y datos breves como un identificador.
Interactivas	Pueden tener ícono, imagen, vibración y, al tocarlas, abrir la app en una sección concreta (`notificationclick`).

Tabla 1. Características de las notificaciones push web.

3. Ventajas y limitaciones

Ventajas

- **Inmediatez:** el aviso llega en segundos a todos los vecinos suscritos.
- **Funcionan con la app cerrada,** igual que en una app nativa, sin publicar nada en tiendas de aplicaciones.
- **Un solo código** para Android, computadora e iPhone.
- **Gratis:** Firebase Cloud Messaging no cobra por mensaje, incluso en el plan Spark.
- **Aumentan la participación:** traen al usuario de vuelta a la app justo donde ocurre algo.

Limitaciones

- **iPhone y iPad:** solo funcionan desde **iOS/iPadOS 16.4** y únicamente si la PWA está **instalada en la pantalla de inicio** (Compartir > Agregar a inicio). En Safari como pestaña normal no hay push.
- **Dependen del permiso:** si el usuario lo bloquea, la app ya no puede volver a pedirlo; debe cambiarlo él en la configuración del sitio.
- **La entrega no está garantizada al instante:** el ahorro de batería del sistema puede retrasar algunos mensajes.
- **Abusar molesta:** demasiadas notificaciones hacen que el usuario las desactive.
- **Necesitan un servidor que envíe:** en el plan gratuito, sin Cloud Functions, el envío se hace con un script (o desde la consola de Firebase).
- **Los tokens caducan** cuando el usuario borra datos del sitio o revoca el permiso; hay que limpiarlos.

4. Aplicaciones prácticas

Las notificaciones push se usan siempre que un evento ocurre fuera del control del usuario y vale la pena avisarle:

Ámbito	Ejemplo de uso	Notificación típica
Seguridad y protección civil	Alertas vecinales, avisos de sismo o de cortes de agua	"Robo reportado en Calle 5 de Mayo"
Mensajería y redes sociales	Versiones web de servicios de chat y redes sociales	"Tienes un mensaje nuevo de Ana"
Comercio electrónico	Tiendas en línea que siguen el estado del pedido	"Tu pedido salió a reparto"
Transporte y viajes	Aerolíneas y servicios de movilidad	"Tu vuelo cambió a la puerta 12"
Noticias	Periódicos digitales	"Última hora: …"
Educación	Plataformas escolares	"La profesora publicó una nueva tarea"
Salud	Recordatorios de citas o medicamentos	"Tu cita es mañana a las 10:00"

Tabla 2. Aplicaciones prácticas de las notificaciones push.

En **AlertaVecinal** se usan para avisar de cada alerta nueva. Al tocar la notificación, la app se abre (o se enfoca si ya estaba abierta) directamente en la alerta correspondiente, que aparece resaltada en la lista.

5. ¿Cómo funciona por dentro?

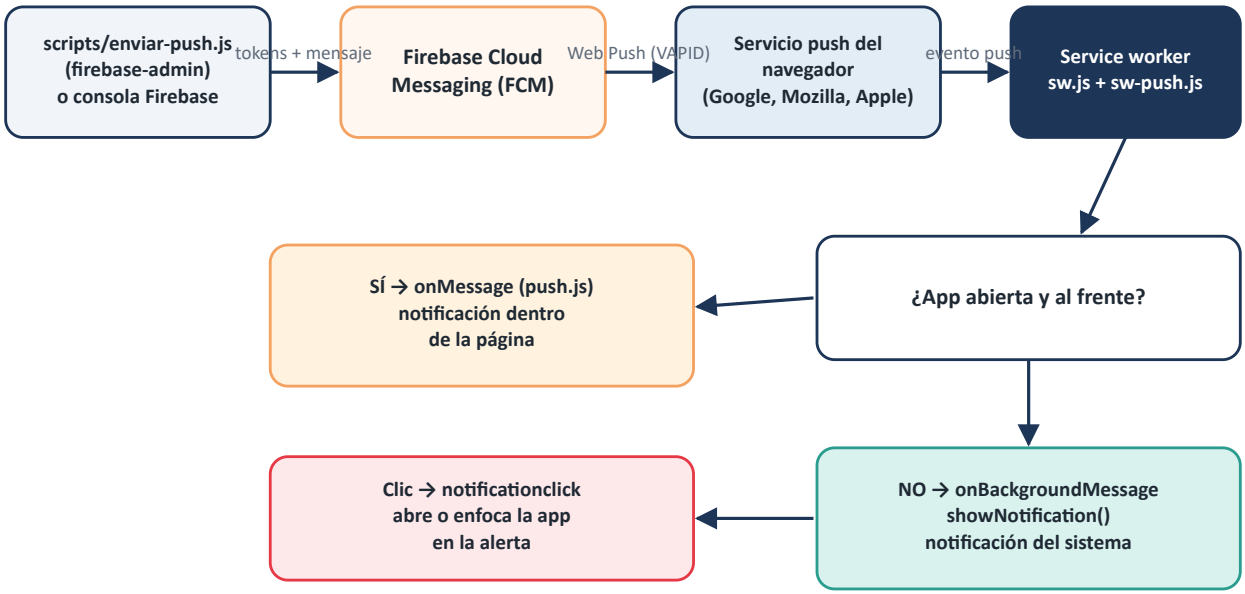


Figura 1. Flujo de una notificación: script → FCM → servicio push del navegador → service worker → notificación. Diagrama elaborado por el equipo.

1. **Envío:** el script `scripts/enviar-push.js` (o la consola de Firebase) entrega a FCM el mensaje y la lista de tokens.
2. **FCM** identifica a qué servicio push pertenece cada token y le reenvía el mensaje firmado con las claves VAPID del proyecto.
3. **Servicio push del navegador:** cada navegador tiene el suyo (el de Google para Chrome y Edge en Android, el de Mozilla para Firefox, el de Apple para Safari). Mantiene una conexión con el dispositivo y le entrega el mensaje.
4. **Service worker:** el navegador lo despierta con el evento `push`. Si la app no está al frente, Firebase llama a `onBackgroundMessage` y se muestra la notificación del sistema con `showNotification()`.
5. **Primer plano:** si la app está abierta y visible, el mensaje llega a `onMessage` en la página y se muestra dentro de ella, para no interrumpir con un aviso del sistema.
6. **Clic:** el evento `notificationclick` abre la app en la alerta o enfoca la pestaña existente.

6. Componentes y tecnologías necesarios

Componente	Qué es	En AlertaVecinal
Service Worker	Script en segundo plano que recibe el push aunque la app esté cerrada.	`sw.js` carga `sw-push.js` con `importScripts`.
Push API	API del navegador para suscribirse a un servicio push y recibir el evento `push`.	La usa internamente el SDK de Firebase Messaging.
Notifications API	Permite pedir permiso y mostrar avisos del sistema.	`Notification.requestPermission()` y `registration.showNotification()`.
Firebase Cloud Messaging	Servicio gratuito de Google que entrega los mensajes a cualquier navegador.	SDK `firebase-messaging` 10.12.2 desde gstatic.
Claves VAPID	Par de claves que identifica al servidor ante los servicios push; la pública va en el cliente.	`FIREBASE_VAPID_KEY` en `js/firebase-config.js`.
Token de registro	Identificador único de cada navegador suscrito.	Se obtiene con `getToken()` y se guarda en la colección `tokens`.
HTTPS	Conexión cifrada, requisito para service worker y push.	Firebase Hosting lo da automáticamente.
firebase-admin	SDK de servidor con permisos totales, autenticado con una cuenta de servicio.	`scripts/enviar-push.js` con `serviceAccountKey.json`.

Tabla 3. Piezas que intervienen en una notificación push.

7. Ejemplo implementado en AlertaVecinal

7.1 Pedir permiso, obtener el token y guardarlo

El permiso se pide al pulsar **Activar notificaciones**, nunca al cargar la página. Después se obtiene el token con la clave VAPID y con el registro de `sw.js` (parámetro `serviceWorkerRegistration`), para no necesitar el archivo `firebase-messaging-sw.js` que Firebase busca por defecto. Al final el token se guarda en Firestore.

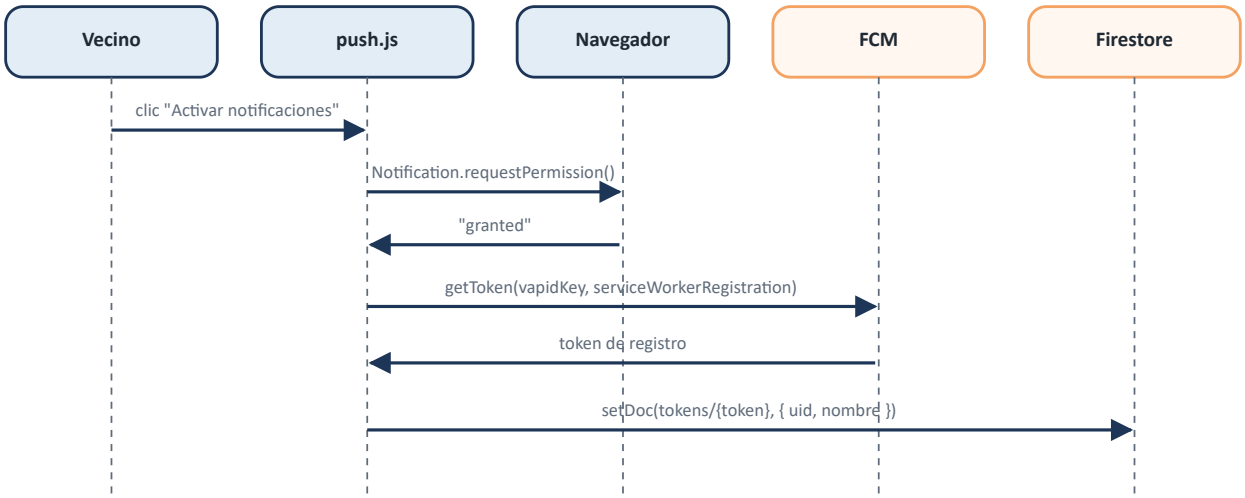


Figura 2. Secuencia para obtener y guardar el token de registro. Diagrama elaborado por el equipo.

Archivo **`js/push.js`**, líneas 45 a 72

```
export async function activarNotificaciones() {
  if (!('Notification' in window)) throw new Error('Tu navegador no soporta notificaciones.');
  // 1) Permiso (el navegador muestra su diálogo).
  const permiso = await Notification.requestPermission();
  pintarEstado();
  if (permiso !== 'granted') throw new Error('No diste permiso para notificaciones.');
  if (esDemo || self.FIREBASE_SIN_CONFIGURAR) {
    throw new Error('Permiso concedido. Para obtener el token, configura Firebase y la clave
VAPID.');
  }
  // 2) Token FCM: identifica a este navegador para poder enviarle mensajes.
  const token = await getToken(await obtenerMensajeria(), {
    vapidKey: self.FIREBASE_VAPID_KEY,
    serviceWorkerRegistration: await registroSW() // usamos sw.js en vez de firebase-messaging-
sw.js
  });
  if (!token) throw new Error('No se pudo obtener el token.');
  tokenActual = token;
  // 3) Guardamos el token en Firestore para que scripts/enviar-push.js lo encuentre.
```

```
  const { usuario, perfil } = window.AV.sesion || {};
  await setDoc(doc(db, 'tokens', token), {
    token,
    uid: usuario?.uid || null,
    nombre: perfil?.nombre || '',
    navegador: navigator.userAgent.slice(0, 150),
    actualizado: serverTimestamp()
  });
  pintarEstado();
  return token;
}
```

7.2 Primer plano: onMessage

Con la app abierta, Firebase entrega el mensaje a `onMessage` y no muestra nada por sí solo. AlertaVecinal lo pinta como una tarjeta dentro de la página con un botón para ver la alerta.

Archivo **`js/push.js`**, líneas 29 a 42

```
async function obtenerMensajeria() {
  if (mensajeria) return mensajeria;
  // isSupported(): FCM necesita service worker, Push API e IndexedDB.
  if (!(await isSupported())) throw new Error('Este navegador no soporta notificaciones push
web.');
  mensajeria = getMessaging(app);
  // Primer plano: el mensaje llega aquí y NO se muestra solo; lo pintamos en la página.
  onMessage(mensajeria, (payload) => {
    console.info('[Push] Mensaje en primer plano:', payload);
    const n = payload.notification || {};
    const d = payload.data || {};
    mostrarEnPagina(n.title || d.titulo || 'AlertaVecinal', n.body || d.cuerpo || '', d.url);
  });
  return mensajeria;
}
```

7.3 Segundo plano: onBackgroundMessage

En el service worker se carga la versión *compat* del SDK (los service workers usan `importScripts`) y la misma configuración del proyecto:

Archivo **`sw-push.js`**, líneas 45 a 50

```
importScripts('js/firebase-config.js');
importScripts(`https://www.gstatic.com/firebasejs/${self.FIREBASE_SDK_VERSION}/firebase-app-
compat.js`);
importScripts(`https://www.gstatic.com/firebasejs/${self.FIREBASE_SDK_VERSION}/firebase-
messaging-compat.js`);
```

```
firebase.initializeApp(self.FIREBASE_CONFIG);
const messaging = firebase.messaging();
```

Archivo **sw-push.js**, líneas 53 a 70

```
messaging.onBackgroundMessage((payload) => {
  console.info('[SW] Push recibido en segundo plano:', payload);
  // Si el mensaje trae "notification" (por ejemplo, el de la consola de Firebase),
  // el SDK ya muestra la notificación solo; no la duplicamos.
  if (payload.notification) return;

  const datos = payload.data || {};
  const titulo = datos.titulo || 'AlertaVecinal';
  return self.registration.showNotification(titulo, {
    body: datos.cuerpo || 'Hay una nueva alerta en tu colonia.',
    icon: 'iconos/icon-192.png',
    badge: 'iconos/badge-96.png',
    tag: datos.alertaId || 'alertavecinal',  // reemplaza notificaciones repetidas
    renotify: true,
    vibrate: [200, 100, 200],
    data: { url: datos.url || (datos.alertaId ? `./?alerta=${datos.alertaId}` : './') }
  });
});
```

7.4 Clic en la notificación: notificationclick

Este manejador se registra antes de cargar Firebase. Busca una ventana abierta de la app; si existe, la lleva a la alerta y la enfoca; si no, abre una nueva.

Archivo **sw-push.js**, líneas 17 a 42

```
self.addEventListener('notificationclick', (event) => {
  event.notification.close();
  // Firebase guarda los datos del mensaje en data.FCM_MSG cuando él muestra la notificación.
  const datos = event.notification.data || {};
  const fcm = datos.FCM_MSG || {};
  const destino = datos.url || fcm.data?.url || fcm.fcmOptions?.link ||
                  fcm.notification?.click_action || './';
  const urlFinal = new URL(destino, self.location.origin).href;

  // Evita que el manejador interno de Firebase abra otra pestaña.
  event.stopImmediatePropagation();

  event.waitUntil((async () => {
    // clients.matchAll(): todas las pestañas/ventanas de la app.
    const ventanas = await self.clients.matchAll({ type: 'window', includeUncontrolled: true });
    for (const ventana of ventanas) {
      if (new URL(ventana.url).origin === self.location.origin && 'focus' in ventana) {
        // Ya hay una ventana abierta: la llevamos a la alerta y la enfocamos.
        await ventana.navigate(urlFinal).catch(() => null);
        return ventana.focus();
      }
```

```
    }
    // No había ninguna: abrimos una nueva.
    return self.clients.openWindow(urlFinal);
  })());
});
```

7.5 Envío desde la terminal: enviar-push.js

Como el plan Spark no incluye Cloud Functions, el envío se hace con un script de Node que se ejecuta en la computadora del administrador. Lee todos los tokens, envía un mensaje de datos con `sendEachForMulticast()` y borra los tokens que ya no sirven.

Archivo **`scripts/enviar-push.js`**, líneas 38 a 72

```
async function main() {
  // 1) Leemos todos los tokens guardados por push.js.
  const snap = await db.collection('tokens').get();
  const tokens = snap.docs.map((d) => d.id);
  if (!tokens.length) {
    console.log('No hay tokens registrados. Activa las notificaciones en la app primero.');
    return;
  }
  console.log(`Enviando a ${tokens.length} dispositivo(s)…`);

  // 2) Mensaje de solo datos ("data"): así el service worker (sw-push.js)
  //    decide cómo mostrarlo en segundo plano y onMessage lo recibe en primer plano.
  const url = alertaId ? `./?alerta=${alertaId}` : './';
  const respuesta = await admin.messaging().sendEachForMulticast({
    tokens,
    data: { titulo, cuerpo, url, alertaId: alertaId || '' },
    webpush: { headers: { Urgency: 'high', TTL: '3600' } }
  });

  console.log(`Enviadas: ${respuesta.successCount} · Fallidas: ${respuesta.failureCount}`);

  // 3) Borramos los tokens que ya no sirven (navegador desinstalado o permiso retirado).
  const invalidos = [];
  respuesta.responses.forEach((r, i) => {
    if (!r.success) {
      const codigo = r.error?.code;
      console.log(` - ${tokens[i].slice(0, 20)}…: ${codigo}`);
      if (codigo === 'messaging/registration-token-not-registered' || codigo ===
'messaging/invalid-registration-token') {
        invalidos.push(db.collection('tokens').doc(tokens[i]).delete());
      }
    }
  });
  await Promise.all(invalidos);
  if (invalidos.length) console.log(`Se borraron ${invalidos.length} token(s) inválidos.`);
}
```

7.6 Pasos para enviar una notificación

Opción A: consola de Firebase

1. En la app, iniciar sesión, pulsar **Activar notificaciones** y aceptar el permiso.
2. Abrir **Demo push** y pulsar **Copiar token**.
3. En la consola de Firebase: *Messaging* > *Nueva campaña* > *Notificaciones*; escribir título y texto.
4. Pulsar **Enviar mensaje de prueba**, pegar el token, pulsar **+** y luego **Probar**.

Opción B: script (requiere `scripts/serviceAccountKey.json`, descargado en *Configuración del proyecto > Cuentas de servicio*):

Terminal (PowerShell)

```
cd scripts
npm install
node enviar-push.js "Alerta" "Robo en calle 5"
```

7.7 Capturas

La página `demo-push.html` muestra el estado del permiso, el estado del service worker, el token con su botón para copiarlo y un botón que lanza una notificación local con `registration.showNotification()`.

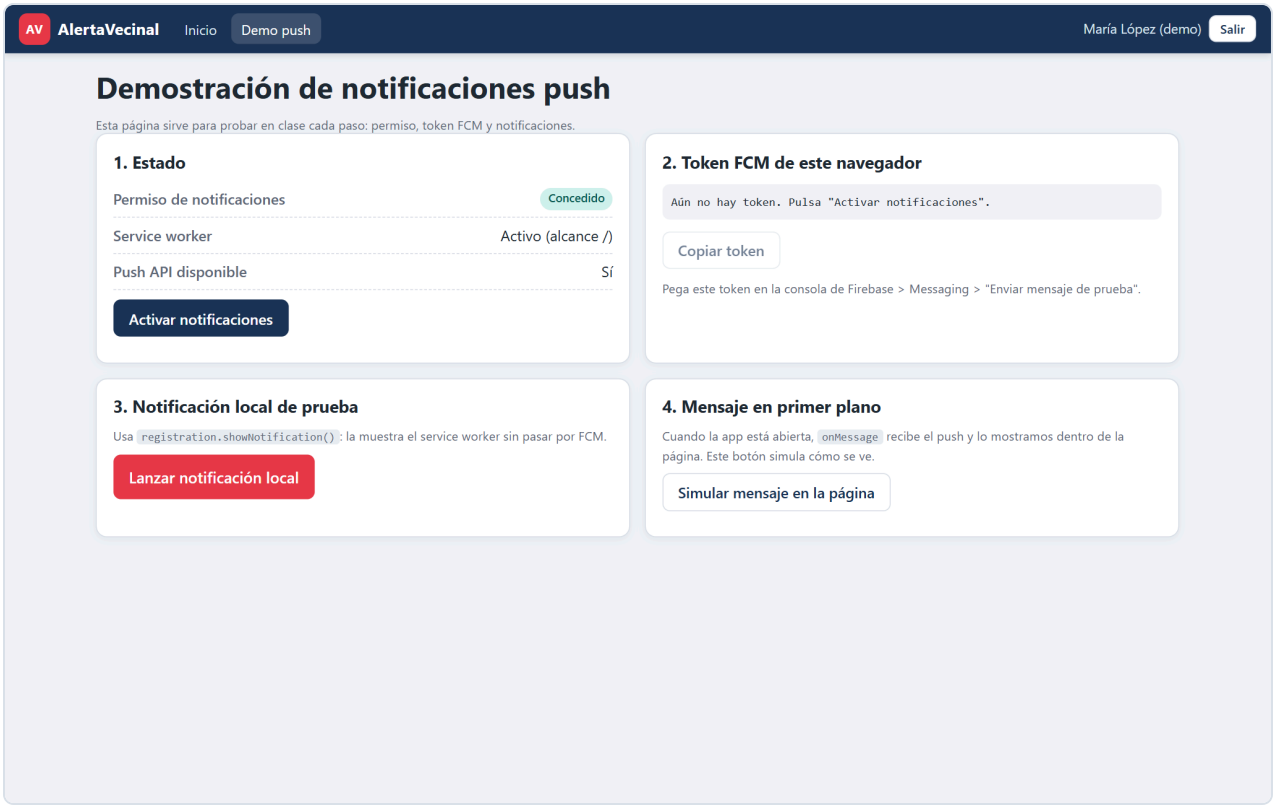


Figura 3. demo-push.html con el permiso concedido y el service worker activo. El token aparece después de configurar las claves de Firebase.

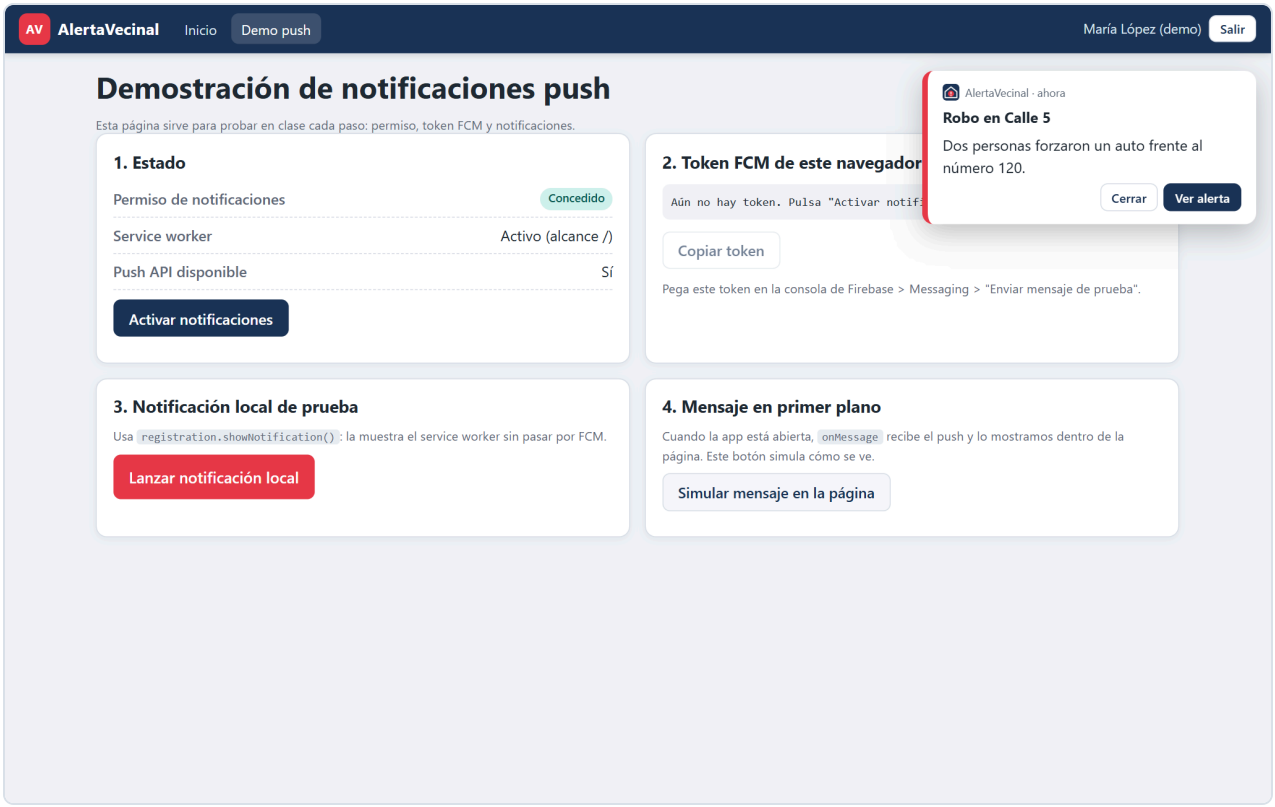


Figura 4. Notificación en primer plano: así muestra la app el mensaje recibido por onMessage.

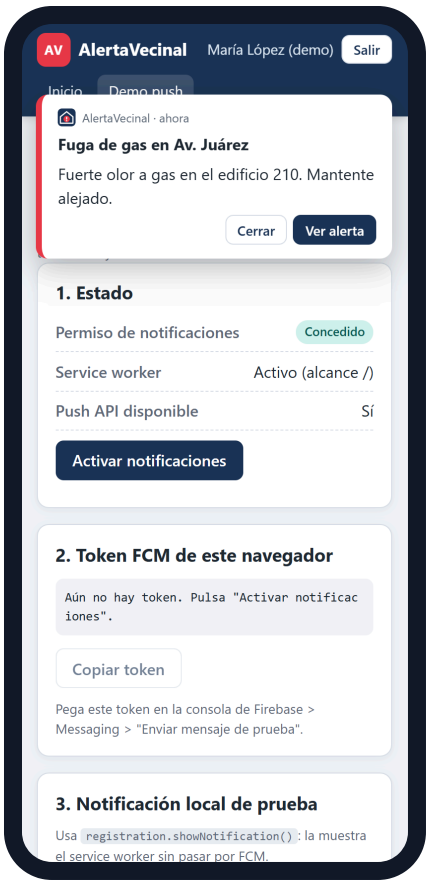


Figura 5. La notificación dentro de la página, en celular.

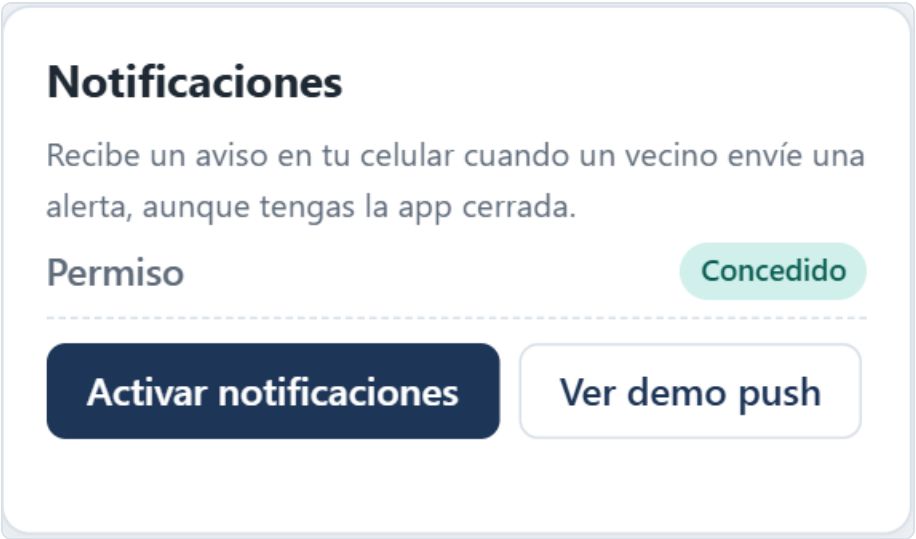


Figura 6. Tarjeta "Notificaciones" del panel, con el botón para pedir permiso.

Nota sobre las capturas: Playwright controla el contenido de la página, pero no puede fotografiar las notificaciones del sistema operativo, porque se dibujan fuera del navegador. Por eso se capturó la notificación dentro de la página. La notificación del sistema sí se generó: durante la prueba se pulsó "Lanzar notificación local" y se comprobó con `registration.getNotifications()` que el service worker la creó con el título **"Prueba de AlertaVecinal"** y el texto **"Robo reportado en Calle 5 de Mayo 120. Toca para ver la alerta."**. En la presentación en clase se verá en la esquina de la pantalla (Windows) o en la barra de notificaciones (Android).

8. Conclusión

Las notificaciones push son lo que convierte a AlertaVecinal en una herramienta de seguridad real: una alerta que nadie ve a tiempo no sirve. Con la PWA, el aviso llega aunque la app esté cerrada, en celular o computadora, y al tocarlo lleva directamente al incidente.

Implementarlas nos enseñó que intervienen muchas piezas (service worker, Push API, Notifications API, FCM, claves VAPID, tokens y HTTPS) y que cada una tiene una responsabilidad clara. También vimos que se puede tener un sistema completo dentro del plan gratuito de Firebase, enviando desde un script en lugar de Cloud Functions. Por último, aprendimos a respetar al usuario: el permiso se pide solo cuando él lo decide, y en iPhone la app debe estar instalada en la pantalla de inicio.

Referencias

Firebase. (s. f.). *Firebase Cloud Messaging*. Google. https://firebase.google.com/docs/cloud-messaging

Firebase. (s. f.). *Receive messages in a JavaScript client*. Google. https://firebase.google.com/docs/cloud-messaging/js/receive

Firebase. (s. f.). *Set up a JavaScript Firebase Cloud Messaging client app*. Google. https://firebase.google.com/docs/cloud-messaging/js/client

Firebase. (s. f.). *Send messages to multiple devices*. Google. https://firebase.google.com/docs/cloud-messaging/send-message

MDN Web Docs. (s. f.). *Notifications API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Notifications_API

MDN Web Docs. (s. f.). *Push API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Push_API

MDN Web Docs. (s. f.). *ServiceWorkerRegistration: showNotification() method*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/ServiceWorkerRegistration/showNotification

Thomson, M., y Beverloo, P. (2017). *Voluntary Application Server Identification (VAPID) for Web Push* (RFC 8292). IETF. https://www.rfc-editor.org/rfc/rfc8292

web.dev. (s. f.). *Push notifications overview*. Google. https://web.dev/articles/push-notifications-overview

WebKit. (2023). *Web Push for Web Apps on iOS and iPadOS*. https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/

URL de la aplicación

La aplicación AlertaVecinal está publicada en Firebase Hosting y se puede abrir desde cualquier navegador moderno, en computadora o en celular, en la siguiente dirección:

[URL_DE_LA_APP]

Para ver la app con datos de ejemplo, sin iniciar sesión, se puede agregar `?demo=1` al final de la dirección.